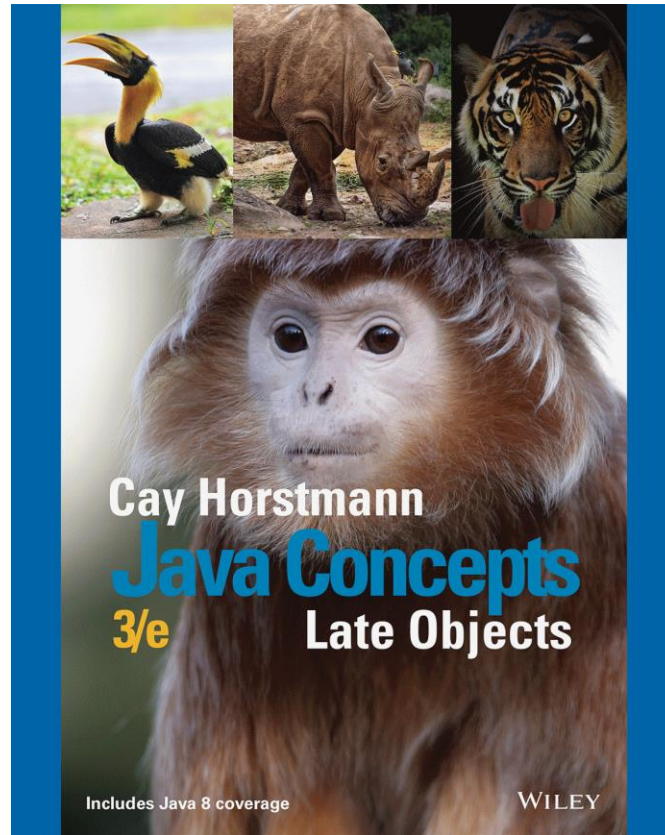


Chapter 4 - Loops



Chapter Goals



- To implement `while`, `for`, and `do` loops
- To hand-trace the execution of a program
- To become familiar with common loop algorithms
- To understand nested loops
- To implement programs that read and process data sets
- To use a computer for simulations

The **while** Loop

- Examples of loop applications
 - Calculating compound interest
 - Simulations, event driven programs...
- Compound interest algorithm (Chapter 1)

Start with a year value of 0, a column for the interest, and a balance of \$10,000.

year	interest	balance
0		\$10,000

Repeat the following steps while the balance is less than \$20,000.

Add 1 to the year value.

Compute the interest as $\text{balance} \times 0.05$ (i.e., 5 percent interest).

Add the interest to the balance.

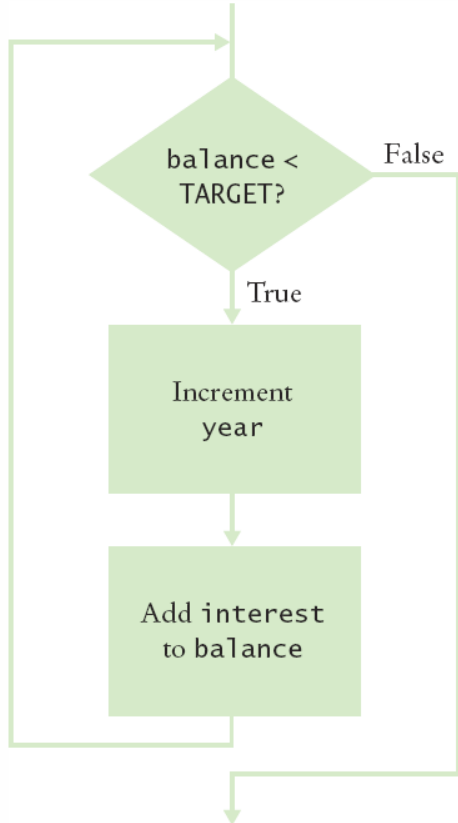
Report the final year value as the answer.

Steps



Planning the **while** Loop

- A loop executes instructions repeatedly while a condition is true



```
while (balance < TARGET)
{
    year++;
    double interest = balance * RATE/100;
    balance = balance + interest;
}
```

Syntax 4.1 **while** Statement

Syntax **while** (*condition*)
 {
 statements
 }

This variable is declared outside the loop and updated in the loop.

If the condition never becomes false, an infinite loop occurs.



See Common Error 4.2.

double balance = 0;

.
. .
. .

while (balance < TARGET)

{

double interest = balance * RATE / 100;

balance = balance + interest;

}

This variable is created in each loop iteration.

Beware of "off-by-one" errors in the loop condition.
 See Common Error 4.3.

Don't put a semicolon here!
 See Common Error 3.1.

These statements are executed while the condition is true.

Lining up braces is a good idea.
See Programming Tip 3.1.

Braces are not required if the body contains a single statement, but it's good to always use them.
 See Programming Tip 3.2.

Execution of the Loop

1 Check the loop condition

balance = 10000

```
while (balance < TARGET)
{
```

The condition is true

4 After 15 iterations

balance = 20789.28

```
while (balance < TARGET)
{
    year++;
    double interest = balance * RATE / 100;
    balance = balance + interest;
}
```

The condition is no longer true

2

year = 15

5 Execute the statement following the loop

balance = 20789.28

year = 15

```
while (balance < TARGET)
{
    year++;
    double interest = balance * RATE / 100;
    balance = balance + interest;
}
System.out.println(year);
```

3

balance = 10500

year = 1

```
year++;
double interest = balance * RATE / 100;
balance = balance + interest;
}
```

DoubleInvestment.java

```
1  /**
2   * This program computes the time required to double an investment.
3   */
4  public class DoubleInvestment
5  {
6      public static void main(String[] args)
7      {
8          final double RATE = 5;
9          final double INITIAL_BALANCE = 10000;
10         final double TARGET = 2 * INITIAL_BALANCE;
11
12         double balance = INITIAL_BALANCE;
13         int year = 0;
14
15         // Count the years required for the investment to double
16
17         while (balance < TARGET)
18         {
19             year++;
20             double interest = balance * RATE / 100;
21             balance = balance + interest;
22         }
23
24         System.out.println("The investment doubled after "
25             + year + " years.");
26     }
27 }
```

Declare and initialize a variable outside of the loop to count `years`

Increment the `years` variable each time through

Program Run

The investment doubled after 15 years.

while Loop Examples (1)

Loop	Output	Explanation
<pre>i = 0; sum = 0; while (sum < 10) { i++; sum = sum + i; Print i and sum; }</pre>	<pre>1 1 2 3 3 6 4 10</pre>	When sum is 10, the loop condition is false, and the loop ends.
<pre>i = 0; sum = 0; while (sum < 10) { i++; sum = sum - i; Print i and sum; }</pre>	<pre>1 -1 2 -3 3 -6 4 -10 . . .</pre>	Because sum never reaches 10, this is an “infinite loop” (see Common Error 4.2 on page 132).
<pre>i = 0; sum = 0; while (sum < 0) { i++; sum = sum - i; Print i and sum; }</pre>	(No output)	The statement <code>sum < 0</code> is false when the condition is first checked, and the loop is never executed.

while Loop Examples (2)

Loop	Output	Explanation
<pre>i = 0; sum = 0; while (sum >= 10) { i++; sum = sum + i; Print i and sum; }</pre>	(No output)	The programmer probably thought, “Stop when the sum is at least 10.” However, the loop condition controls when the loop is executed, not when it ends (see Common Error 4.1 on page 132).
<pre>i = 0; sum = 0; while (sum < 10) ; { i++; sum = sum + i; Print i and sum; }</pre>	(No output, program does not terminate)	Note the semicolon before the {. This loop has an empty body. It runs forever, checking whether <code>sum < 0</code> and doing nothing in the body.

Self Check 4.1

How many years does it take for the investment to triple? Modify the program and run it.

Self Check 4.2

If the interest rate is 10 percent per year, how many years does it take for the investment to double? Modify the program and run it.

Self Check 4.3

Modify the program so that the balance after each year is printed. How did you do that?

Self Check 4.4

Suppose we change the program so that the condition of the `while` loop is

```
while (balance <= TARGET)
```

What is the effect on the program? Why?

Self Check 4.5

What does the following loop print?

```
int n = 1;
while (n < 100)
{
    n = 2 * n;
    System.out.print(n + " ");
}
```

Common Error

- Don't think "Are we there yet?"
 - The loop body will only execute if the test condition is **True**
 - "Are we there yet?" should continue if **False**
 - If `bal` should grow until it reaches `TARGET`
 - Which version will execute the loop body?

```
while (bal < TARGET)
{
    year++;
    interest = bal * RATE;
    bal = bal + interest;
}
```

```
while (bal >= TARGET)
{
    year++;
    interest = bal * RATE;
    bal = bal + interest;
}
```

Common Error

- Infinite Loops

- The loop body will execute until the test condition becomes **False**
- What if you forget to update the test variable?
 - `bal` is the test variable (`TARGET` doesn't change)
 - You will loop forever! (or until you stop the program)

```
while (bal < TARGET)
{
    year++;
    interest = bal * RATE;
}
```

Common Error

- Off-by-One Errors

- A 'counter' variable is often used in the test condition
- Your counter can start at 0 or 1, but programmers often start a counter at 0
- If I want to paint all 5 fingers, when I am done?

▪ **Start at 0, use <**

```
int finger = 0;
final int FINGERS = 5;
while (finger < FINGERS)
{
    // paint finger
    finger++;
}
```

▪ **Start at 1, use <=**

```
int finger = 1;
final int FINGERS = 5;
while (finger <= FINGERS)
{
    // paint finger
    finger++;
}
```

Problem Solving: Hand-Tracing

- Example: Calculate the sum of digits (1+7+2+9)
 - Make columns for key variables (n, sum, digit)
 - Examine the code and number the steps
 - Set variables to state before loop begins

n	sum	digit
1729	0	



```
int n = 1729;
int sum = 0;
while (n > 0)
{
    int digit = n % 10;
    sum = sum + digit;
    n = n / 10;
}
System.out.println(sum);
```

Tracing Sum of Digits

- Start executing loop body statements changing variable values on a new line
- Cross out *values* in previous line

n	sum	digit
1729	0	
	9	9

```
int n = 1729;
int sum = 0;
while (n > 0)
{
    int digit = n % 10;
    sum = sum + digit;
    n = n / 10;
}
System.out.println(sum);
```

Tracing Sum of Digits

- Continue executing loop statements changing variables
- $1729 / 10$ leaves 172 (no remainder)

n	sum	digit
1729	0	
172	9	9

```
int n = 1729;
int sum = 0;
while (n > 0)
{
    int digit = n % 10;
    sum = sum + digit;
    n = n / 10;
}
System.out.println(sum);
```

Tracing Sum of Digits

- Test condition. If true, execute loop again
 - Variable `n` is 172, Is $172 > 0$?, True!
- Make a new line for the second time through and update variables

n	sum	digit
1729	0	
172	9	9
17	11	2

```
int n = 1729;
int sum = 0;
while (n > 0)
{
    int digit = n % 10;
    sum = sum + digit;
    n = n / 10;
}
System.out.println(sum);
```

Tracing Sum of Digits

- Third time through
 - Variable `n` is 17 which is still greater than 0
- Execute loop statements and update variables

n	sum	digit
1729	0	
172	9	9
17	11	2
1	18	7

```
int n = 1729;
int sum = 0;
while (n > 0)
{
    int digit = n % 10;
    sum = sum + digit;
    n = n / 10;
}
System.out.println(sum);
```

Tracing Sum of Digits

- Fourth loop iteration:
 - Variable n is 1 at start of loop. $1 > 0$? True
 - Executes loop and changes variable n to 0 ($1/10 = 0$)

n	sum	digit
1729	0	
172	9	9
17	11	2
1	18	7
0	19	1

```
int n = 1729;
int sum = 0;
 while (n > 0)
{
    int digit = n % 10;
    sum = sum + digit;
    n = n / 10;
}
System.out.println(sum);
```

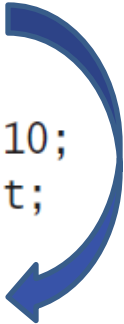
Tracing Sum of Digits

- Because n is 0, the expression $(n > 0)$ is False
- Loop body is not executed
 - Jumps to next statement after the loop body
- Finally prints the sum!

n	sum	digit
1729	0	
172	9	9
17	11	2
1	18	7
0	19	1



```
int n = 1729;
int sum = 0;
while (n > 0)
{
    int digit = n % 10;
    sum = sum + digit;
    n = n / 10;
}
System.out.println(sum);
```



Self Check 4.6

Hand-trace the following code, showing the value of `n` and the output.

```
int n = 5;
while (n >= 0)
{
    n--;
    System.out.print(n);
}
```

Self Check 4.7

Hand-trace the following code, showing the value of `n` and the output. What potential error do you notice?

```
int n = 1;
while (n <= 3)
{
    System.out.print(n + ", ");
    n++;
}
```

Self Check 4.8

Hand-trace the following code, assuming that `a` is 2 and `n` is 4. Then explain what the code does for arbitrary values of `a` and `n`.

```
int r = 1;
int i = 1;
while (i <= n)
{
    r = r * a;
    i++;
}
```

Self Check 4.9

Trace the following code. What error do you observe?

```
int n = 1;
while (n != 50)
{
    System.out.println(n);
    n = n + 10;
}
```

Self Check 4.10

The following pseudocode is intended to count the number of digits in the number n :

```
count = 1
```

```
temp = n
```

```
while (temp > 10)
```

```
    Increment count.
```

```
    Divide temp by 10.0.
```

Trace the pseudocode for $n = 123$ and $n = 100$. What error do you find?

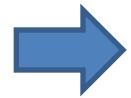
The **for** Loop

- Use a **for** loop when you:
 - Can use an integer counter variable
 - Have a constant increment (or decrement)
 - Have a fixed starting and ending value for the counter
- Use a **for** loop when a value runs from a starting point to an ending point with a constant increment or decrement

```
int i = 5; // initialize
while (i <= 10) // test
{
    sum = sum + 1;
    i++; // update
}
```

```
for (int i = 5; i <= 10; i++)
{
    sum = sum + i;
}
```

Execution of a **for** Loop



1 Initialize counter

counter = 1

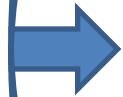
```
for (counter = 1; counter <= 10; counter++)  
{  
    System.out.println(counter);  
}
```



2 Check condition

counter = 1

```
for (counter = 1; counter <= 10; counter++)  
{  
    System.out.println(counter);  
}
```



3 Execute loop body

counter = 1

```
for (counter = 1; counter <= 10; counter++)  
{  
    System.out.println(counter);  
}
```



4 Update counter

counter = 2

```
for (counter = 1; counter <= 10; counter++)  
{  
    System.out.println(counter);  
}
```

Syntax 4.2 **for** Statement

Syntax **for** (*initialization*; *condition*; *update*)
 {
 statements
 }

These three
expressions should be related.
See Programming Tip 4.1.

This initialization
happens once
before the loop starts.

The condition is
checked before
each iteration.

This update is
executed after
each iteration.

```
for (int i = 5; i <= 10; i++)  
{  
    sum = sum + i;  
}
```

The variable *i*
is defined only in this
for loop.

This loop executes 6 times.
See Programming Tip 4.3.

When To Use a **for** Loop?

- Yes, a **while** loop can do everything a **for** loop can do
- Programmers like it because it is concise
 - Initialization
 - Condition
 - Update
- All on one line!

In general, the for loop:

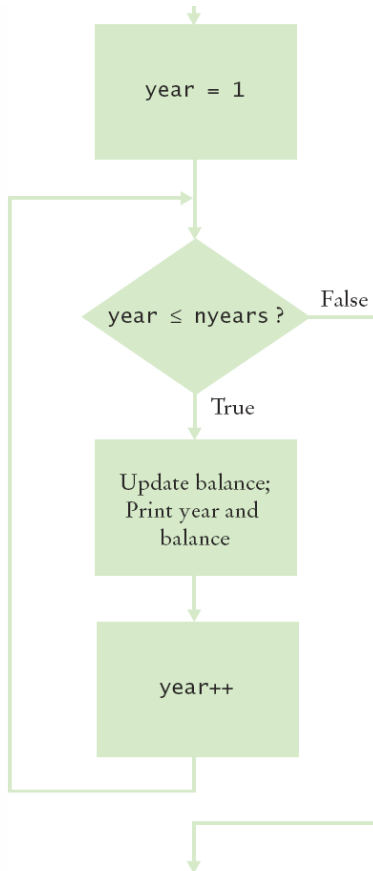
```
for (initialization; condition; update)  
{  
    statements  
}
```

has exactly the same effect as the while loop:

```
initialization;  
while (condition)  
{  
    statements  
    update  
}
```

Planning a **for** Loop

- Print the balance at the end of each year for a number of years



Year	Balance
1	10500.00
2	11025.00
3	11576.25
4	12155.06
5	12762.82

```
for (int year = 1; year <= nyears; year++)  
{  
    Update balance.  
    Print year and balance.  
}
```

InvestmentTable.java

- Setup variables
- Get input
- Loop
 - Calc
 - Output

```
1  import java.util.Scanner;
2
3  /**
4   * This program prints a table showing the growth of an investment.
5   */
6  public class InvestmentTable
7  {
8      public static void main(String[] args)
9      {
10         final double RATE = 5;
11         final double INITIAL_BALANCE = 10000;
12         double balance = INITIAL_BALANCE;
13
14         System.out.print("Enter number of years: ");
15         Scanner in = new Scanner(System.in);
16         int nyears = in.nextInt();
17
18         // Print the table of balances for each year
19
20         for (int year = 1; year <= nyears; year++)
21         {
22             double interest = balance * RATE / 100;
23             balance = balance + interest;
24             System.out.printf("%4d %10.2f\n", year, balance);
25         }
26     }
27 }
```

Good Examples of for Loops

Table 2 for Loop Examples

Loop	Values of i	Comment
for (i = 0; i <= 5; i++)	0 1 2 3 4 5	Note that the loop is executed 6 times. (See Programming Tip 4.4 on page 153.)
for (i = 5; i >= 0; i--)	5 4 3 2 1 0	Use i-- for decreasing values.
for (i = 0; i < 9; i = i + 2)	0 2 4 6 8	Use i = i + 2 for a step size of 2.
for (i = 0; i != 9; i = i + 2)	0 2 4 6 8 10 12 14 ... (infinite loop)	You can use < or <= instead of != to avoid this problem.
for (i = 1; i <= 20; i = i * 2)	1 2 4 8 16	You can specify any rule for modifying i, such as doubling it in every step.
for (i = 0; i < str.length(); i++)	0 1 2 ... until the last valid index of the string str	In the loop body, use the expression str.charAt(i) to get the ith character.

for Loop Variable Scope

- Scope is the 'lifetime' of a variable.
- When 'x' is declared in the `for` statement:
- 'x' exists only inside the 'block' of the for loop { }

```
for( int x = 1; x < 10; x = x + 1) {  
    // steps to do inside the loop  
    // You can use 'x' anywhere in this box  
}  
  
if (x > 100)    // Error! x is out of scope!
```

- Solution: Declare 'x' outside the `for` loop

```
int x;  
for(x = 1; x < 10; x = x + 1)
```

Self Check 4.11

Write the `for` loop of the `InvestmentTable.java` program as a `while` loop.

Self Check 4.12

How many numbers does this loop print?

```
for (int n = 10; n >= 0; n--)  
{  
    System.out.println(n);  
}
```

Self Check 4.13

Write a `for` loop that prints all even numbers between 10 and 20 (inclusive).

Self Check 4.14

Write a `for` loop that computes the sum of the integers from 1 to n .

Self Check 4.15

How would you modify the `for` loop of the `InvestmentTable.java` program to print all balances until the investment has doubled?

Programming Tip

- Use `for` loops for their intended purposes only
 - Increment (or decrement) by a constant value
 - Do not update the counter inside the body
 - Update in the third section of the header

```
for (int counter = 1; counter <= 100; counter++)  
{  
    if (counter % 10 == 0) // Skip values divisible by 10  
    {  
        counter++; // Bad style: Do NOT update the counter inside loop  
    }  
    System.out.println(counter);  
}
```

- Most counters start at one 'end' (0 or 1)
 - Many programmers use an integer named `i` for 'index' or 'counter' variable in `for` loops

Programming Tip

- Choose loop bounds that match your task
 - `for` loops establish lower and upper bounds on an index
 - When the same conditional operator is used for both bounds, the bounds are called **symmetric**

```
1 <= i <= 10
```

- When different conditional operators are used for both bounds, the bounds are called **asymmetric**

```
1 <= i < 10
```

- Both kinds of bounds can be appropriate in loops
- Asymmetric bounds work well with strings

```
for(int i = 0; i < str.length(); i++)
```

Programming Tip

- Count Iterations
 - Many bugs are 'off by one' issues
 - One too many or one too few
- How many posts are there?
- How many pairs of rails are there?

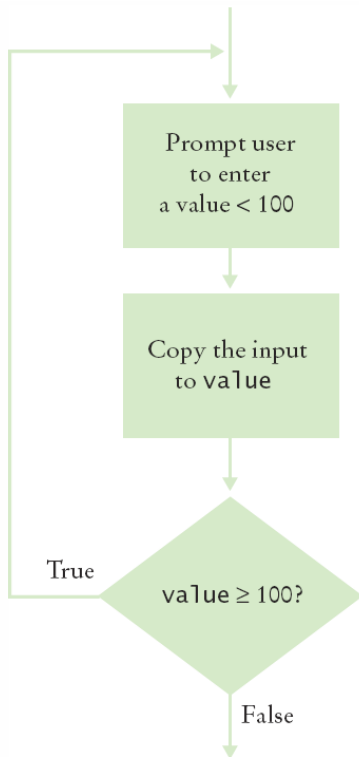
```
final int RAILS = 5;  
for (int i = 1; i < RAILS; i++)  
{  
    System.out.println("Painting rail " + i);  
}
```

```
Painting rail 1  
Painting rail 2  
Painting rail 3  
Painting rail 4
```



▪The **do** Loop

- Use a **do** loop when you want to:
 - Execute the body at least once
 - Test the condition **AFTER** your first loop



```
int i = 1; // initialize
final int FINGERS = 5;
do
{
    // paint finger
    i++; // update
}
while (i <= FINGERS); // test
```

Note the semicolon at the end!

do Loop Example

- User Input Validation:
 - Range check a value entered
 - User must enter something to validate first!

```
int value;  
do  
{  
    System.out.println("Enter an integer < 100: ");  
    value = in.nextInt();  
}  
while (value >= 100);  // test
```

Self Check 4.16

Suppose that we want to check for inputs that are at least 0 and at most 100. Modify the `do` loop in this section for this check.

Self Check 4.17

Rewrite the input check `do` loop using a `while` loop. What is the disadvantage of your solution?

Self Check 4.18

Suppose Java didn't have a `do` loop. Could you rewrite any `do` loop as a `while` loop?

```
}
```

Self Check 4.19

Write a `do` loop that reads integers and computes their sum. Stop when reading the value 0.

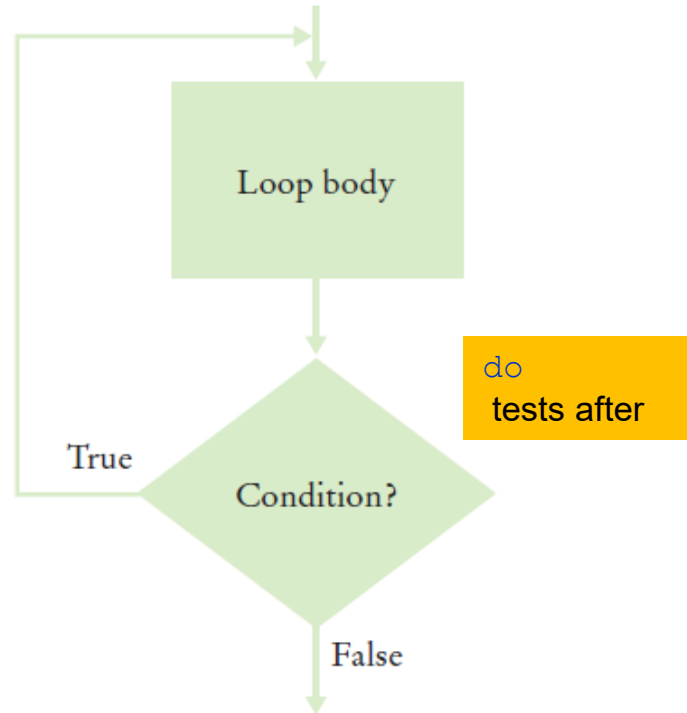
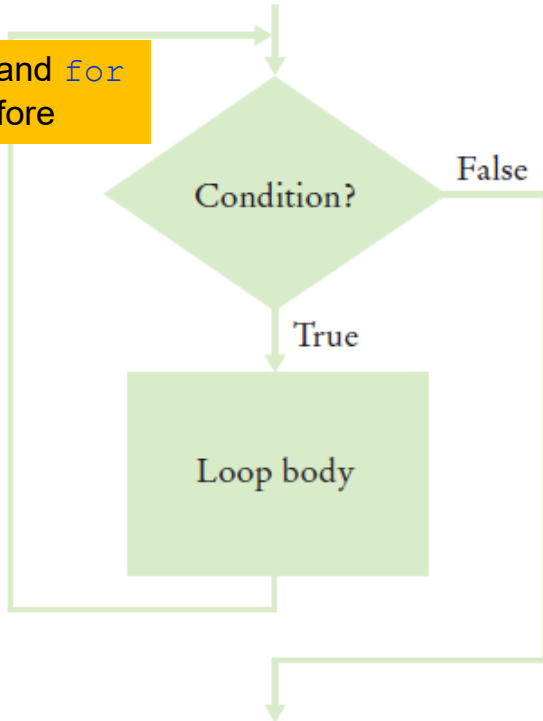
Self Check 4.20

Write a `do` loop that reads integers and computes their sum. Stop when reading a zero or the same value twice in a row. For example, if the input is 1 2 3 4 4, then the sum is 14 and the loop stops.

Programming Tip

- Flowcharts for loops
 - To avoid 'spaghetti code', never have an arrow that points inside the loop body

`while` and `for`
test before



`do`
tests after

Processing Sentinel Values

- A sentinel value denotes the end of a data set, but it is not part of the data
- Sentinel values are often used:
 - When you don't know how many items are in a list, use a 'special' character or value to signal no more items.
 - For numeric input of positive numbers, it is common to use the value **-1**:



```
salary = in.nextDouble();  
while (salary != -1)  
{  
    sum = sum + salary;  
    count++;  
    salary = in.nextDouble();  
}
```

Averaging a Set of Values

- Declare and initialize a 'sum' variable to 0
- Declare and initialize a 'count' variable to 0
- Declare and initialize an 'input' variable to 0
- Prompt user with instructions
- Loop until sentinel value is entered
 - Save entered value to input variable
 - If input is not -1 (sentinel value)
 - Add input to sum variable
 - Add 1 to count variable
- Make sure you have at least one entry before you divide!
 - Divide sum by count and output. Done!

SentinelDemo.java (1)

```
8 public static void main(String[] args)
9 {
10     double sum = 0;
11     int count = 0;
12     double salary = 0;
13     System.out.print("Enter salaries, -1 to finish: ");
14     Scanner in = new Scanner(System.in);
15
16     // Process data until the sentinel is entered
17
18     while (salary != -1)
19     {
20         salary = in.nextDouble();
21         if (salary != -1)
22         {
23             sum = sum + salary;
24             count++;
25         }
26     }
27 }
```

Outside the `while` loop, declare and initialize variables to use

Since `salary` is initialized to 0, the `while` loop statements will be executed

Input new `salary` and compare to sentinel

Update running `sum` and `count` to average later

SentinelDemo.java (2)

```
28 // Compute and print the average
29
30 if (count > 0) Prevent divide by 0
31 {
32     double average = sum / count;
33     System.out.println("Average salary: " + average);
34 }
35 else
36 {
37     System.out.println("No data");
38 }
39 }
40 }
```

Calculate and output the average salary using `sum` and `count` variables

Program Run

```
Enter salaries, -1 to finish: 10 10 40 -1
Average salary: 20
```

Boolean Variables and Sentinels

- A boolean variable can be used to control a loop
 - Sometimes called a 'flag' variable

```
System.out.print("Enter salaries, -1 to finish: ");
boolean done = false;
while (!done)
{
    value = in.nextDouble();
    if (value == -1)
    {
        done = true;
    }
    else
    {
        // Process value
    }
}
```

To Input Any Numeric Value...

- When valid values can be positive or negative
 - You cannot use -1 (or any other number) as a sentinel
- One solution is to use a non-numeric sentinel
 - But Scanner's `in.nextDouble` will fail!
 - Use Scanner's `in.hasNextDouble` first
 - Returns a boolean: `true` (all's well) or `false` (not a number)
 - Then use `in.nextDouble` if `true`

```
System.out.print("Enter values, Q to quit: ");  
while (in.hasNextDouble())  
{  
    value = in.nextDouble();  
    // Process value  
}
```

Self Check 4.21

What does the `SentinelDemo.java` program print when the user immediately types `-1` when prompted for a value?

Self Check 4.22

Why does the `SentinelDemo.java` program have *two* checks of the form
`salary != -1`

Self Check 4.23

What would happen if the declaration of the `salary` variable in `SentinelDemo.java` was changed to

```
double salary = -1;
```

Self Check 4.24

In the last example of this section, we prompt the user “Enter values, Q to quit.” What happens when the user enters a different letter?

Self Check 4.25

What is wrong with the following loop for reading a sequence of values?

```
System.out.print("Enter values, Q to quit: ");
do
{
    double value = in.nextDouble();
    sum = sum + value;
    count++;
}
while (in.hasNextDouble());
```

Storyboards

- One useful problem solving technique is the use of storyboards to model user interaction. It can help answer:
 - What information does the user provide, and in which order?
 - What information will your program display, and in which format?
 - What should happen when there is an error?
 - When does the program quit?

Storyboard Example

- Goal: Converting a sequence of values
 - Will require a loop and some variables
 - Handle one conversion each time through the loop

Converting a Sequence of Values

What unit do you want to convert from? *cm*

What unit do you want to convert to? *in*

Enter values, terminated by zero ————— Allows conversion of multiple values

30

30 *cm* = 11.81 *in*

100

100 *cm* = 39.37 *in*

0

What unit do you want to convert from?

Format makes clear what got converted

What Can Go Wrong?

- Unknown unit types
 - How do you spell centimeters and inches?
 - What other conversions are available?
 - Solution:
 - Show a list of the acceptable unit types

From unit (in, ft, mi, mm, cm, m, km, oz, lb, g, kg, tsp, tbsp, pint, gal): **cm**

To unit: **in**

No need to list the units again

What Else Can Go Wrong?

- How does the user quit the program?

Exiting the Program

From unit (in, ft, mi, mm, cm, m, km, oz, lb, g, kg, tsp, tbsp, pint, gal): **cm**

To unit: **in**

Enter values, terminated by zero

30

30 cm = 11.81 in

0

More conversions (y, n)? **n**

(Program exits)

Sentinel triggers the prompt to exit

- Storyboards help you plan a program
 - Knowing the flow helps you structure your code

Self Check 4.26

Provide a storyboard panel for a program that reads a number of test scores and prints the average score. The program only needs to process one set of scores. Don't worry about error handling.

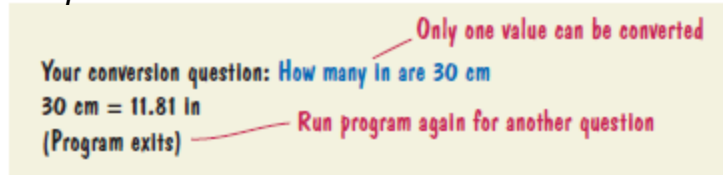
Self Check 4.27

Google has a simple interface for converting units. You just type the question, and you get the answer.

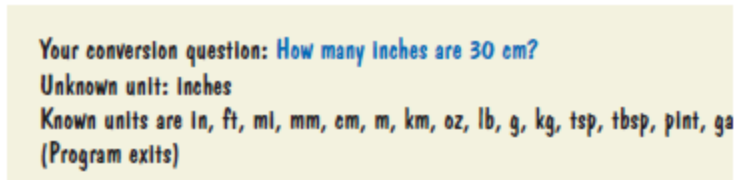


Make storyboards for an equivalent interface in a Java program. Show a scenario in which all goes well, and show the handling of two kinds of errors.

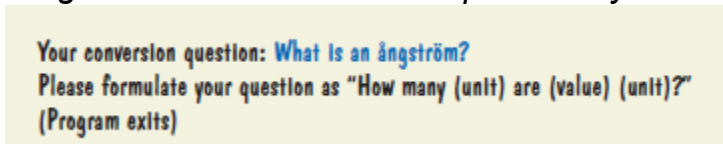
Answer: *Simple conversion*



Unknown unit



Program doesn't understand question syntax



Self Check 4.28

Consider a modification of the program in Self Check 26. Suppose we want to drop the lowest score before computing the average. Provide a storyboard for the situation in which a user only provides one score.

Self Check 4.29

What is the problem with implementing the following storyboard in Java?

Computing Multiple Averages

Enter scores: 90 80 90 100 80

The average is 88

Enter scores: 100 70 70 100 80

The average is 84

Enter scores: -1

(Program exits)

-1 is used as a sentinel to exit the program

Self Check 4.30

Produce a storyboard for a program that compares the growth of a \$10,000 investment for a given number of years under two interest rates.

Common Loop Algorithms

- 1: Sum and Average Value
- 2: Counting Matches
- 3: Finding the First Match
- 4: Prompting until a match is found
- 5: Maximum and Minimum
- 6: Comparing Adjacent Values

Sum and Average Examples

▪Sum of Values

- Initialize total to 0
- Use while loop with sentinel

```
double total = 0;
while (in.hasNextDouble())
{
    double input = in.nextDouble();
    total = total + input;
}
```

▪Average of Values

- Use Sum of Values
- Initialize count to 0
 - Increment per input
- Check for count 0
 - Before divide!

```
double total = 0;
int count = 0;
while (in.hasNextDouble())
{
    double input = in.nextDouble();
    total = total + input;
    count++;
}
double average = 0;
if (count > 0)
{
    average = total / count;
}
```

Counting Matches

- Counting Matches
- Initialize count to 0
- Use a `for` loop
- Add to count per match



```
int upperCaseLetters = 0;
for (int i = 0; i < str.length(); i++)
{
    char ch = str.charAt(i);
    if (Character.toUpperCase(ch))
    {
        upperCaseLetters++;
    }
}
```

Finding the First Match

- Initialize boolean sentinel to false
- Initialize position counter to 0
 - First char in String
- Use a compound conditional in loop

```
boolean found = false;
char ch;
int position = 0;
while (!found &&
       position < str.length())
{
    ch = str.charAt(position);
    if (Character.isLowerCase(ch))
    {
        found = true;
    }
    else { position++; }
}
```



Prompt Until a Match Is Found

- Initialize `boolean` flag to `false`
- Test sentinel in `while` loop
 - Get input, and compare to range
 - If input is in range, change flag to `true`
 - Loop will stop executing

```
boolean valid = false;
double input;
while (!valid)
{
    System.out.print("Please enter a positive value < 100: ");
    input = in.nextDouble();
    if (0 < input && input < 100) { valid = true; }
    else { System.out.println("Invalid input."); }
}
```

Maximum and Minimum

- Get first input value
 - This is the **largest** (or **smallest**) that you have seen so far!
- Loop while you have a valid number (non-sentinel)
 - Get another input value
 - Compare new input to **largest** (or **smallest**)
 - Update **largest** (or **smallest**), if necessary

```
double largest = in.nextDouble();
while (in.hasNextDouble())
{
    double input = in.nextDouble();
    if (input > largest)
    {
        largest = input;
    }
}
```

```
double smallest = in.nextDouble();
while (in.hasNextDouble())
{
    double input = in.nextDouble();
    if (input < smallest)
    {
        smallest = input;
    }
}
```

Comparing Adjacent Values

- Get first input value
- Use `while` to determine if there are more to check
 - Copy input to previous variable
 - Get next value into input variable
 - Compare input to previous, and output if same

```
double input = in.nextDouble();
while (in.hasNextDouble())
{
    double previous = input;
    input = nextDouble();
    if (input == previous)
    {
        System.out.println("Duplicate input");
    }
}
```



Self Check 4.31

What total is computed when no user input is provided in the algorithm in Section 4.7.1?

Self Check 4.32

How do you compute the total of all positive inputs?

Self Check 4.33

What are the values of `position` and `ch` when no match is found in the algorithm in Section 4.7.3?

.

Self Check 4.34

What is wrong with the following loop for finding the position of the first space in a string?

```
boolean found = false;
for (int position = 0; !found && position < str.length();
    position++)
{
    char ch = str.charAt(position);
    if (ch == ' ') { found = true; }
}
```

Self Check 4.35

How do you find the position of the *last* space in a string?

Self Check 4.36

What happens with the algorithm in Section 4.7.6 when no input is provided at all?
How can you overcome that problem?

Steps to Writing a Loop

Planning:

1. Decide what work to do inside the loop
2. Specify the loop condition
3. Determine loop type
4. Setup variables before the first loop
5. Process results when the loop is finished
6. Trace the loop with typical examples

Coding:

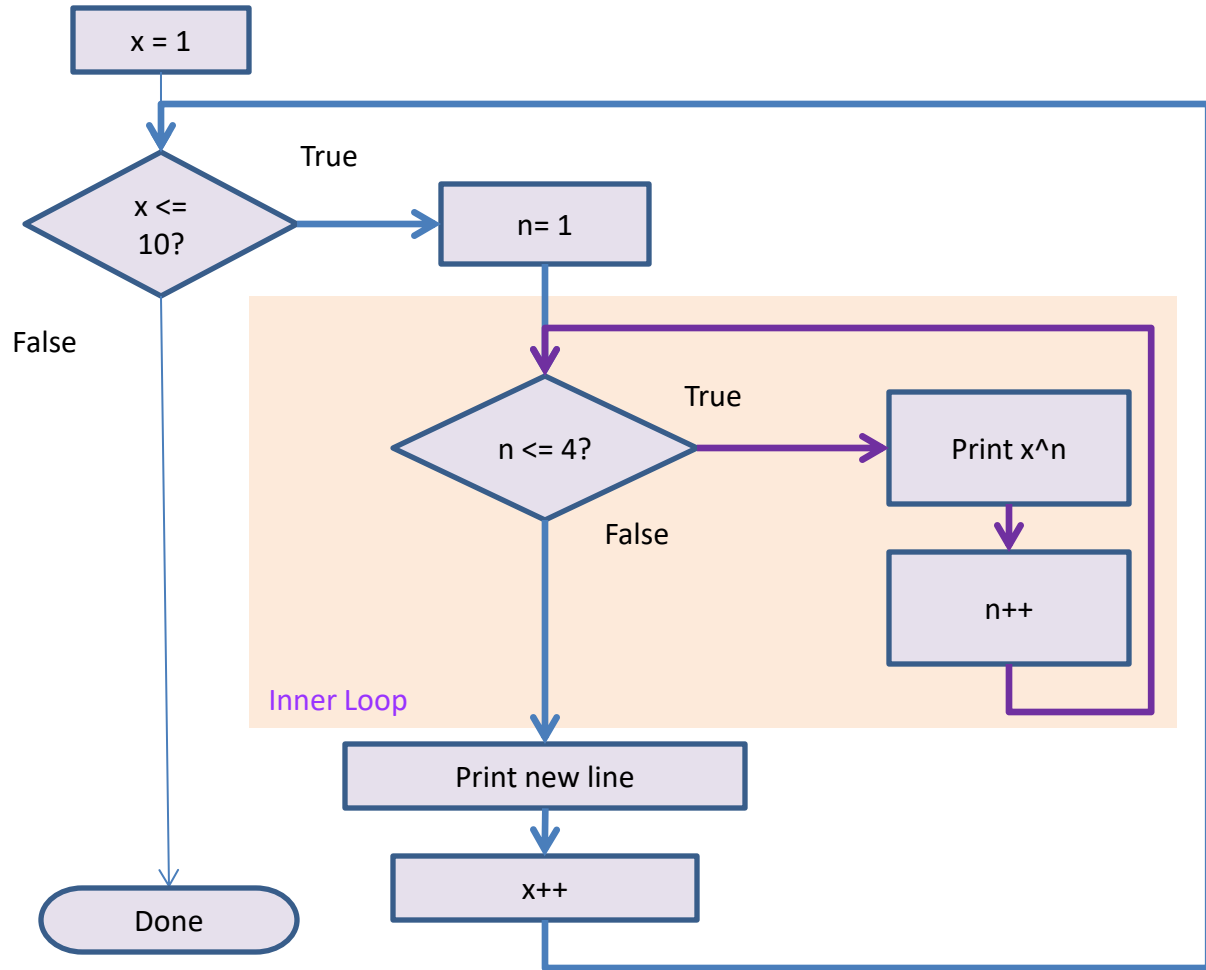
7. Implement the loop in Java

Nested Loops

- How would you print a table with rows and columns?
 - Print top line (header)
 - Use a `for` loop
 - Print table body...
 - How many rows?
 - How many columns?
 - Loop per row
 - Loop per column

x^1	x^2	x^3	x^4
1	1	1	1
2	4	8	16
3	9	27	81
...	...	...	...
10	100	1000	10000

Flowchart of a Nested Loop



PowerTable.java

```
1  /**
2   * This program prints a table of powers of x.
3   */
4  public class PowerTable
5  {
6      public static void main(String[] args)
7      {
8          final int NMAX = 4;
9          final double XMAX = 10;
10
11         // Print table body
12
13         for (double x = 1; x <= XMAX; x++)
14         {
15             // Print table row
16
17             for (int n = 1; n <= NMAX; n++)
18             {
19                 System.out.printf("%10.0f", Math.pow(x, n));
20             }
21             System.out.println();
22         }
23     }
24 }
```

1	2	3	4
x	x	x	x
1	1	1	1
2	4	8	16
3	9	27	81
4	16	64	256
5	25	125	625
6	36	216	1296
7	49	343	2401
8	64	512	4096
9	81	729	6561
10	100	1000	10000

Body of outer loop

Body of inner loop

Nested Loop Examples (1)

```
for (i = 1; i <= 3; i++)  
{  
    for (j = 1; j <= 4; j++) { print "*" }  
    print new line  
}
```

```
****  
****  
****
```

Prints 3 rows of 4 asterisks each.

```
for (i = 1; i <= 4; i++)  
{  
    for (j = 1; j <= 3; j++) { print "*" }  
    print new line  
}
```

```
***  
***  
***  
***
```

Prints 4 rows of 3 asterisks each.

```
for (i = 1; i <= 4; i++)  
{  
    for (j = 1; j <= i; j++) { print "*" }  
    print new line  
}
```

```
*  
**  
***  
****
```

Prints 4 rows of lengths 1, 2, 3, and 4.

Nested Loop Examples (2)

```
for (i = 1; i <= 3; i++)  
{  
    for (j = 1; j <= 5; j++)  
    {  
        if (j % 2 == 0) { Print "*" }  
        else { Print "-" }  
    }  
    System.out.println();  
}
```

```
-*-*-  
-*-*-  
-*-*-
```

Prints asterisks in even columns, dashes in odd columns.

```
for (i = 1; i <= 3; i++)  
{  
    for (j = 1; j <= 5; j++)  
    {  
        if (i % 2 == j % 2) { Print "*" }  
        else { Print " " }  
    }  
    System.out.println();  
}
```

```
* * *  
* *  
* * *
```

Prints a checkerboard pattern.

Self Check 4.37

Why is there a statement `System.out.println()` ; in the outer loop but not in the inner loop of the `PowerTable.java` program?

Self Check 4.38

How would you change the program to display all powers from x^0 to x^5 ?

Self Check 4.39

If you make the change in Self Check 38, how many values are displayed?

Self Check 4.40

What do the following nested loops display?

```
for (int i = 0; i < 3; i++)  
{  
    for (int j = 0; j < 4; j++)  
    {  
        System.out.print(i + j);  
    }  
    System.out.println();  
}
```

Self Check 4.41

Write nested loops that make the following pattern of brackets:

```
[] [] [] []
```

```
[] [] [] []
```

```
[] [] [] []
```

Problem Solving: Gallery6.java

```
1 public class Gallery6
2 {
3     public static void main(String[] args)
4     {
5         final int MAX_WIDTH = 720;
6         final int GAP = 10;
7         final int PICTURES = 20;
8
9         Picture pic = new Picture();
10        pic.load("picture1.jpg");
11        int x = 0;
12        int y = 0;
13        int maxY = 0;
14
15        for (int i = 2; i <= 20; i++)
16        {
17            maxY = Math.max(maxY, y + pic.getHeight());
18            Picture previous = pic;
19            pic = new Picture();
20            pic.load("picture" + i + ".jpg");
21            x = x + previous.getWidth() + GAP;
22            if (x + pic.getWidth() >= MAX_WIDTH)
23            {
24                x = 0;
25                y = maxY + GAP;
26            }
27            pic.move(x, y);
28        }
29    }
30 }
```

Self Check 4.42

Suppose you are asked to find all words in which no letter is repeated from a list of words. What simpler problem could you try first?

Self Check 4.43

You need to write a program for DNA analysis that checks whether a substring of one string is contained in another string. What simpler problem can you solve first?

Self Check 4.44

You want to remove “red eyes” from images and are looking for red circles. What simpler problem can you start with?

Self Check 4.45

Consider the task of finding numbers in a string. For example, the string “In 1987, a typical personal computer cost \$3,000 and had 512 kilobytes of RAM.” has three numbers. Break this task down into a sequence of simpler tasks.

Random Numbers and Simulations

- Games often use random numbers to make things interesting
 - Rolling Dice
 - Spinning a wheel
 - Pick a card
- A simulation usually involves looping through a sequence of events
 - Days
 - Events

RandomDemo.java

```
1  /**
2   * This program prints ten random numbers between 0 and 1.
3   */
4  public class RandomDemo
5  {
6      public static void main(String[] args)
7      {
8          for (int i = 1; i <= 10; i++)
9          {
10             double r = Math.random();
11             System.out.println(r);
12         }
13     }
14 }
```

Program Run

```
0.2992436267816825
0.43860176045313537
0.7365753471168408
0.6880250194282326
0.1608272403783395
0.5362876579988844
0.3098705906424375
0.6602909916554179
0.1927951611482942
0.8632330736331089
```

Simulating Die Tosses

- Goal
 - Get a random integer between 1 and 6

```
1  /**
2   * This program simulates tosses of a pair of dice.
3   */
4  public class Dice
5  {
6      public static void main(String[] args)
7      {
8          for (int i = 1; i <= 10; i++)
9          {
10             // Generate two random numbers between 1 and 6
11
12             int d1 = (int) (Math.random() * 6) + 1;
13             int d2 = (int) (Math.random() * 6) + 1;
14             System.out.println(d1 + " " + d2);
15         }
16         System.out.println();
17     }
18 }
```



Program Run

```
5 1
2 1
1 2
5 1
1 2
6 4
4 4
6 1
6 3
5 2
```

Self Check 4.46

How do you simulate a coin toss with the `Math.random()` method?

Self Check 4.47

How do you simulate the picking of a random playing card?

Self Check 4.48

Why does the loop body in `Dice.java` call `Math.random()` twice?

Self Check 4.49

In many games, you throw a pair of dice to get a value between 2 and 12. What is wrong with this simulated throw of a pair of dice?

```
int sum = (int) (Math.random() * 11) + 2;
```

Self Check 4.50

How do you generate a random floating-point number ≥ 0 and < 100 ?

While Loop: Key Steps

- 1. Initialize:** Set up any variables needed before the loop starts.
- 2. Condition Check:** Before each iteration, check if the loop condition is True.
- 3. Execute Block:** If the condition is True, run the code inside the loop.
- 4. Update:** Change variables as needed inside the loop to eventually make the condition False.
- 5. Repeat or Exit:** Go back to step 2. If the condition is False, exit the loop.

For Loop: Key Steps

- 1. Initialize:** Set up the loop variable (often done automatically).
- 2. Condition/Range:** Define the range or sequence to iterate over.
- 3. Execute Block:** Run the code inside the loop for each value in the sequence.
- 4. Update:** The loop variable is updated automatically after each iteration.
- 5. Repeat or Exit:** Continue until all items in the sequence are processed, then exit.

Key Steps of a Do Loop

- 1. Initialize:** Set up any variables needed before the loop starts.
- 2. Execute Block:** Run the code inside the loop **at least once** (this is the main difference from while and for loops).
- 3. Check Condition:** After executing the block, check if the loop condition is True.
- 4. Repeat or Exit:**
 - If the condition is True, go back to step 2 and repeat.
 - If the condition is False, exit the loop.

Key Differences:

- ❑ **While:** Checks condition before running the block.
- ❑ **For:** Designed for a known number of iterations.
- ❑ **Do:** Always runs the block at least once, checks condition after.

Summary of the key steps for while, for, and do loops

Feature / Step	While Loop	For Loop	Do Loop (do...while)
Initialization	Before the loop	At the start of the loop	Before the loop
Condition Check	Before each iteration	Before each iteration	After each iteration
Execution Block	Runs only if condition is True	Runs for each item in range/sequence	Always runs at least once
Update Step	Manual (inside loop body)	Automatic (built into loop structure)	Manual (inside loop body)
Exit Condition	When condition becomes False	When range/sequence is exhausted	When condition becomes False
Use Case	Unknown number of iterations	Known number of iterations	At least one guaranteed execution
Syntax Example	<code>while (condition) { ... }</code>	<code>for (init; condition; update) { ... }</code>	<code>do { ... } while (condition);</code>

Summary

- ❑ A loop executes instructions repeatedly while a condition is true.
- ❑ An off-by-one error is a common error when programming loops. Think through simple test cases to avoid this type of error.
- ❑ Hand-tracing is a simulation of code execution in which you step through instructions and track the values of the variables.
- ❑ Hand-tracing can help you understand how an unfamiliar algorithm works.
- ❑ Hand-tracing can show errors in code or pseudocode.
- ❑ The **while loop** is a fundamental control structure in programming that allows a block of code to be executed **repeatedly** as long as a specified condition remains **true**. It's especially useful when you don't know in advance how many times the loop should run.
- ❑ The for loop is used when a value runs from a starting point to an ending point with a constant increment or decrement.
- ❑ The do loop is appropriate when the loop body must be executed at least once.
- ❑ A sentinel value denotes the end of a data set, but it is not part of the data.
- ❑ You can use a Boolean variable to control a loop. Set the variable to true before entering the loop, then set it to false to leave the loop.
- ❑ Use input redirection to read input from a file. Use output redirection to capture program output in a file.
- ❑ A storyboard consists of annotated sketches for each step in an action sequence.
- ❑ Developing a storyboard helps you understand the inputs and outputs that are required for a program.
- ❑ To compute an average, keep a total and a count of all values.
- ❑ To count values that fulfill a condition, check all values and increment a counter for each match.